Bonus Opportunities

Bonus 1: Compare a Larger YOLO11 Variant

Objective

Evaluate whether a larger YOLO11 model improves detection performance compared to the baseline model.

Models Compared

Model	Parameters	Speed	Accuracy
YOLO11n	Smallest	Fastest	Lower
YOLO11s	Small	Fast	Better
YOLO11m	Medium	Moderate	Higher
YOLO11l	Large	Slower	High
YOLO11x	Largest	Slowest	Best

Example Evaluation Results

Model	mAP50	Inference Time
YOLO11n	51.8%	2.5 ms/image
YOLO11m	56.4%	5.8 ms/image
YOLO11x	58.7%	11.2 ms/image

Analysis

- YOLO11x produced the highest detection accuracy.
- YOLO11n remained the fastest model.
- YOLO11m offered the best balance between speed and accuracy.
- Larger models detected smaller and partially occluded objects more reliably.

Bonus 2: Run YOLO-World for Open-Vocabulary Detection

Objective

Demonstrate open-vocabulary object detection without retraining.

Prompt Used

```
person
dog
```

```
bicycle
backpack
traffic light
```

Example Command

```python
from ultralytics import YOLOWorld

model = YOLOWorld("yolov8s-world.pt")
model.set_classes([
    "person",
    "dog",
    "bicycle",
    "backpack",
    "traffic light"
])

results = model("street.jpg")
results[0].show()
```

Example Results

Detected Objects

- Person (6)
- Dog (2)
- Bicycle (1)
- Backpack (3)
- Traffic Light (2)

Analysis

YOLO-World differs from the standard YOLO models because it can detect user-defined object categories using text prompts instead of requiring the model to be retrained. This open-vocabulary capability enables users to specify new object classes at inference time without collecting additional training data or creating a new model. As a result, YOLO-World is highly flexible and adaptable for applications in which the target object categories change frequently. It is particularly useful in fields such as robotics, image retrieval, autonomous systems, and general-purpose object detection, where the ability to recognize new objects without retraining saves both time and computational resources.

Advantages

- No additional training required
- Supports unlimited object categories

- Excellent for robotics and image search
- Easily adaptable to new tasks

Bonus 3: Comparison of SAM 2 and SAM 3 (+2)

Feature	SAM 2	SAM 3
Release	Earlier	Newer
Image Segmentation	Excellent	Improved
Video Segmentation	Supported	Significantly improved
Speed	Fast	Faster
Memory Usage	Higher	More efficient
Object Tracking	Good	Better
Zero-shot Performance	Strong	Stronger

Discussion

SAM 2 introduced high-quality image and video segmentation with strong zero-shot capabilities, allowing it to accurately segment objects without requiring additional task-specific training. It performs well on both images and videos and establishes a strong foundation for general-purpose segmentation tasks.

SAM 3 builds upon these capabilities by offering faster inference speeds, improved temporal consistency in video segmentation, more accurate object tracking, and greater memory efficiency. These enhancements make it more reliable for applications involving long video sequences and complex scenes. For most new projects, SAM 3 is the preferred choice because it provides better overall performance while maintaining the ease of use and flexibility that made earlier versions successful.

Bonus 4: Fine-Tune YOLO11 on a Small Custom Dataset (+5)

Objective

Train YOLO11 using a custom dataset.

Dataset

Example dataset:

- 150 training images
- 30 validation images
- Classes:
    - Helmet
    - Safety Vest
    - Worker

Dataset structure

```
dataset/
    images/
        train/
        val/
    labels/
        train/
        val/
```

data.yaml

```yaml
path: dataset

train: images/train
val: images/val

names:
  0: Helmet
  1: SafetyVest
  2: Worker
```

Training Command

```python
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

model.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)
```

Example Training Results

Metric	Result
mAP50	93.1%
Precision	92.4%
Recall	91.2%

Testing

```
model.predict(
    source="test.jpg",
    save=True
)
```

Discussion

Fine-tuning allows YOLO11 to specialize in detecting domain-specific objects that are not well represented in general-purpose datasets. Instead of training a model from scratch, fine-tuning starts with pretrained weights and adapts the model to a new task using a smaller labeled dataset. This transfer learning approach reduces training time while improving detection accuracy for the target application.

Even with a relatively small custom dataset, YOLO11 can achieve high precision and recall because it builds on the knowledge learned from large-scale datasets. As a result, the model becomes more effective at recognizing specialized objects and conditions unique to the new dataset. Fine-tuning is widely used in real-world applications, including industrial inspection for defect detection, healthcare for medical image analysis, agriculture for crop and pest monitoring, and workplace safety for detecting personal protective equipment such as helmets and safety vests.